

Governança Unity Catalog

Um app onde usuários de negócio comentam e classificam colunas do Unity Catalog — e a decisão de design que faz isso ser seguro: quando a ação roda com a identidade do próprio usuário, e quando roda com a identidade do robô.

app: **governanca-unity-catalog**

stack: **Streamlit** · **databricks-sdk**

estado interno: **apps.governanca_unity_catalog_<env>**

deploy: **manual, fora do Asset Bundle**

app.py: **1815 linhas, 1 módulo**

01 Visão geral 02 Identidade: você ou o robô
03 O que acontece quando você edita uma coluna 04 Isolamento DEV/PROD
05 Cadastros & auditoria 06 Recursos Databricks 07 Notas e observações

01 Visão geral

É um app **Streamlit** (não AppKit) de um módulo só: usuários de negócio navegam catálogo → schema → tabela → coluna e aplicam comentários e *tags governadas* do Unity Catalog. Diferente dos apps AppKit do time, não usa Lakebase, Volumes ou Model Serving — fala só com SQL Warehouse e duas APIs de identidade/governança. Deploy é manual (`databricks apps deploy`), de propósito fora do Asset Bundle e da pasta que a pipeline de PROD limpa a cada execução.

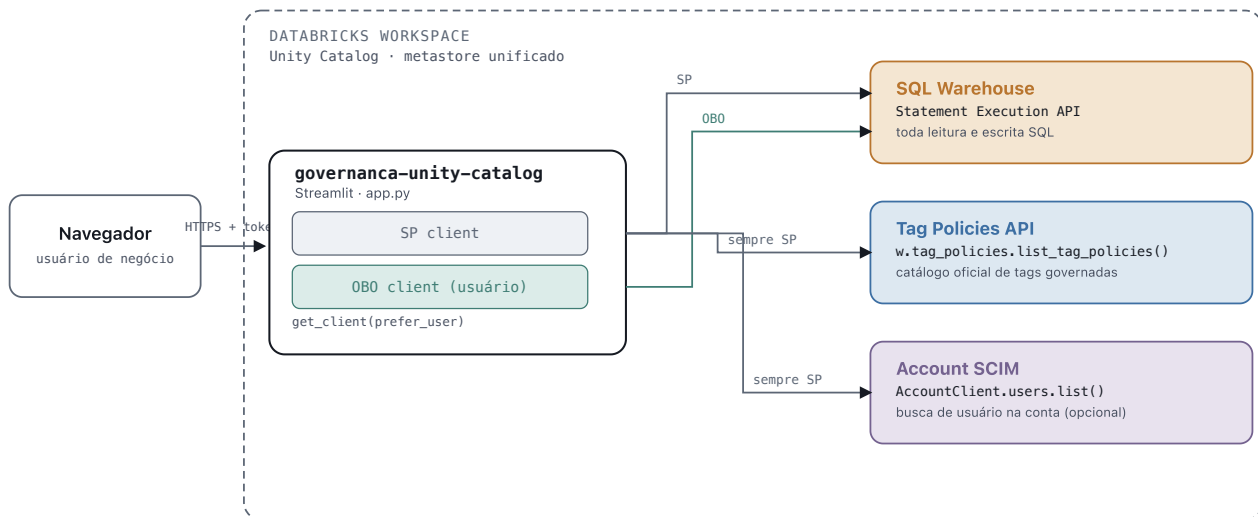


Fig. 1 — O app mantém dois clientes do SDK: um com a identidade do **service principal** e um com a identidade **on-behalf-of do usuário** logado (o token vem no header x-forwarded-access-token, injetado pelo runtime do Databricks Apps). O SQL Warehouse recebe chamadas dos dois; Tag Policies API e Account SCIM só aceitam o service principal.

02 Identidade: quando é você, quando é o robô

Essa é a decisão de design que sustenta o app inteiro. A regra, tal como está no código (função `get_client(prefer_user)`, chamada em cada operação):

OPERAÇÃO	CHAMADA	IDENTIDADE	POR QUÊ
Navegar catálogo/schema/tabela	SHOW CATALOGS/SCHEMAS · information_schema	OBO	o usuário só deve ver o que ele mesmo enxerga
Ler amostra de dados / comentário atual	SELECT ... LIMIT 5	OBO	idem — leitura respeita grants do usuário
Aplicar / remover tag	ALTER TABLE ... SET/UNSET TAGS	OBO	só quem tem APPLY TAG + ASSIGN consegue — a UC decide, o app não precisa
Listar tags governadas (catálogo oficial)	w.tag_policies.list_tag_policies()	SP	é um catálogo global da conta, não um grant por objeto
Comentar tabela / coluna	COMMENT ON TABLE/COLUMN ...	SP	ninguém recebe MODIFY (destravaria escrita de dados) — o robô empresta o dele, com portão na frente (§03)
Cadastros (domínios/sub/stewards/permisões)	CRUD em apps.governanca_unitary_catalog_<env>	SP	não são objetos do catálogo de negócio — controlados por papel interno, não por grant de UC
Log de auditoria	INSERT log_comentarios / log_tags	SP	escrita interna, best-effort, nunca deve travar a governança
Buscar usuário (workspace / conta)	w.users.list() · AccountClient.users.list()	SP	diretório de identidade, não dado do usuário-alvo

0 portão de acesso na frente da única escrita como SP

Como ninguém tem MODIFY, comentar só é seguro se o app checar, com o token do *próprio usuário*, se aquele objeto aparece no `information_schema` dele — ou seja, se ele já teria pelo menos USE/SELECT. Qualquer exceção é tratada como acesso negado (*fail-closed*): não existe caminho de "erro = permite".

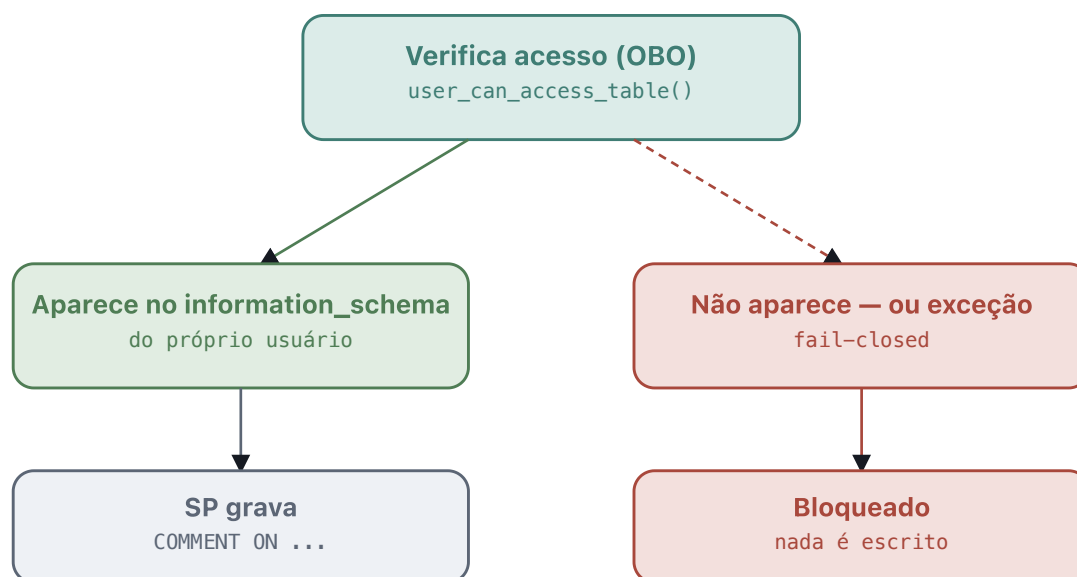


Fig. 2 — A checagem roda com o token do usuário; a escrita, se liberada, roda com o robô. Isso é o oposto do padrão mais comum ("escreve com o que tiver permissão") — aqui a permissão é sempre checada como o usuário, mesmo quando quem escreve é o service principal.

03 0 que acontece quando você edita uma coluna

Do clique em "Salvar e Aplicar Governança" até o log de auditoria — 8 passos, cada um com sua própria identidade (a cor da barra à esquerda), incluindo o desvio de bloqueio do portão de acesso.

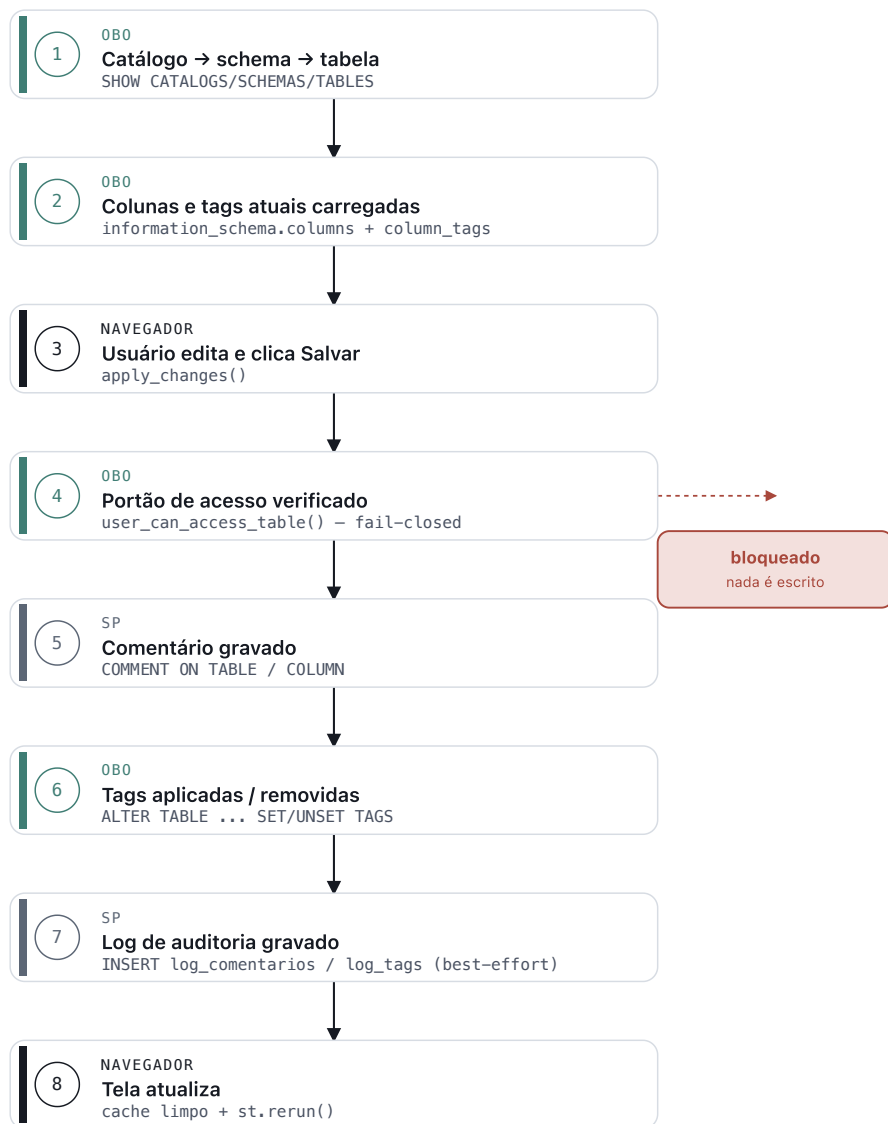


Fig. 3 — O portão (passo 4) roda de novo, independentemente, tanto para o comentário quanto para as tags — não é uma checagem única reaproveitada. O log (passo 7) é melhor-esforço: uma falha ao gravar o log nunca desfaz nem bloqueia a governança que já foi aplicada.

04 Isolamento DEV/PROD

Duas instâncias completamente separadas do app (SPs, warehouses e URLs diferentes) apontando para o **mesmo metastore**. O filtro que você vê na tela (nomes de schema terminando em `_dev`) é só conveniência visual — o limite real são os grants de cada service principal.

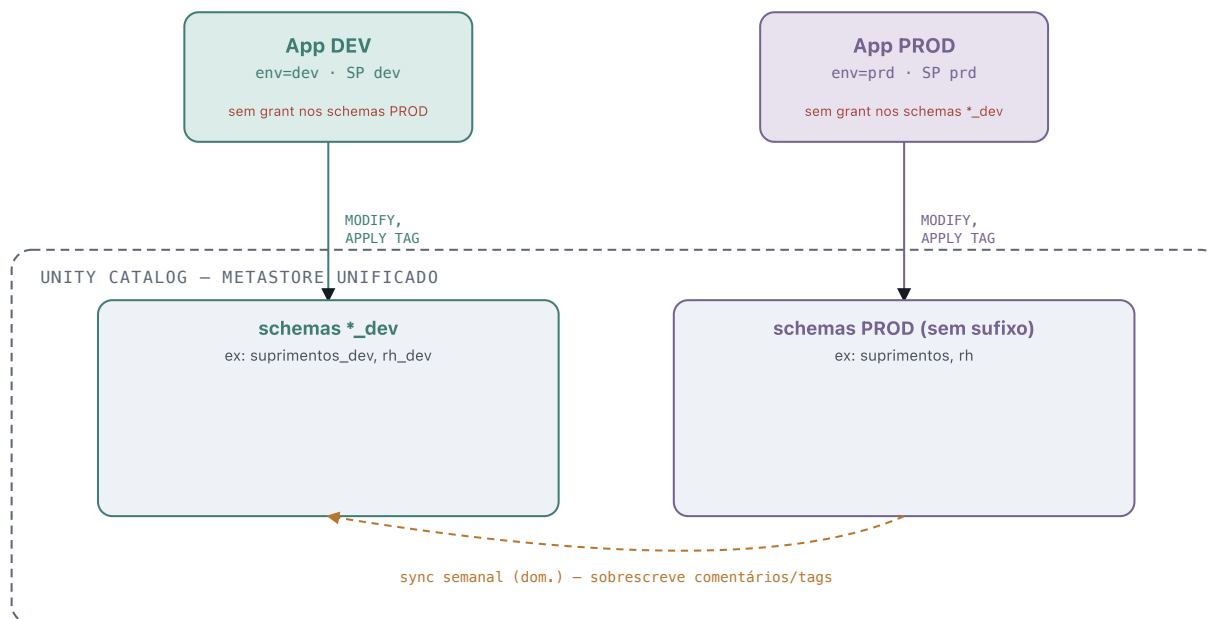


Fig. 4 — Mesmo que o filtro de UX da instância DEV fosse contornado, a escrita falharia com `PERMISSION_DENIED`: o SP de cada ambiente só tem grant no seu próprio grupo de schemas. A seta âmbar é um job separado (notebooks/utils/sync_prod_to_dev.py) que reconcilia comentários/tags de PROD para DEV semanalmente — qualquer governança feita só na instância DEV é sobrescrita nesse ciclo.

05 Cadastros & auditoria

Estado próprio do app — nada disso é Lakebase, são 6 tabelas Delta comuns no catálogo apps, criadas de forma idempotente a cada início de processo. Domínios/sub-domínios/stewards formam uma hierarquia; permissões e os dois logs são independentes dela.

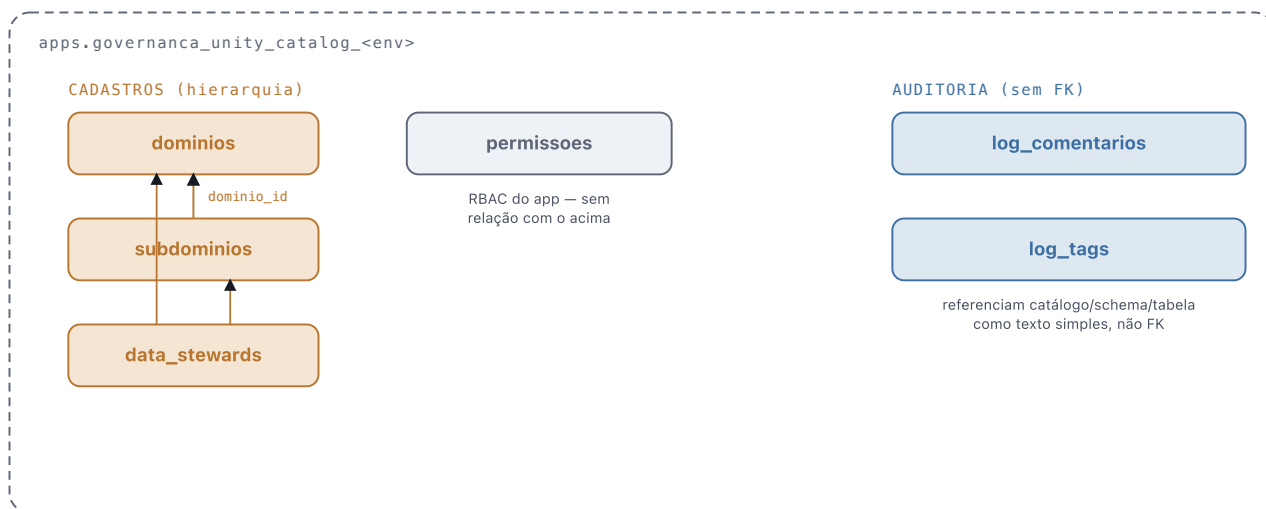


Fig. 5 — Nenhuma dessas tabelas tem FK/unicidade no nível do Delta/UC: unicidade de nome, de steward por domínio+sub-domínio+e-mail, e a regra "sempre um admin" são só código de aplicação (padrão `INSERT ... WHERE NOT EXISTS` para evitar corrida).

TABELA	COLUNAS PRINCIPAIS	REGRA DE NEGÓCIO
dominios	id, nome, descricao	nome único; delete bloqueado se houver sub-domínio ou steward filho
subdominios	id, dominio_id, nome, descricao	nome único por domínio; delete bloqueado se houver steward filho
data_stewards	id, dominio_id, subdominio_id, nome, email	único por (domínio, sub-domínio, e-mail)
permissoes	id, email, papel, ver_logs, ver_cadastros	papel ∈ admin/editor/leitor; não deixa apagar o último admin
log_comentarios	usuario, executor, acao, objeto, catalogo, db_schema, tabela, coluna, comentario_anterior/novo, ambiente, criado_em	append-only; usuario = quem pediu, executor = 'service_principal' fixo
log_tags	usuario, executor, acao, catalogo, db_schema, tabela, coluna, tag_chave, valor_anterior/novo, ambiente, criado_em	append-only; executor = 'on_behalf_of_user' fixo

Papéis (RBAC dos cadastros)

Controla só o menu e a edição de cadastros/logs — **não** tem efeito nenhum sobre quem pode comentar/tagueiar dados de negócio (aquilo é grant de UC + o portão de acesso, seção 02).

Usuário sem registro em permissões entra como leitor por padrão.

● admin — vê e edita tudo, único que gerencia Usuários & Permissões

● editor — edita cadastros, se `ver_cadastros=true`

● leitor — padrão, somente leitura

06 Recursos Databricks

Só três pontos de contato — mais enxuto que um app AppKit, porque a própria governança nativa do Unity Catalog faz a maior parte do trabalho de autorização.

SQL Warehouse

toda leitura e escrita SQL

Um único funil (`run_query` / `run_exec`) para navegação, amostras, DDL de comentário/tag e todo o CRUD dos cadastros.

dev eb0314aa7f7a27c2

prd 99d8bd5c236bde9b

Tag Policies API

catálogo oficial de tags

Fonte única de chaves e valores permitidos — o app nunca inventa opção de tag, sempre lê da política.

chamada w.tag_policies

identidade sempre SP

Account SCIM

busca de usuário (opcional)

Só ativa se `DATABRICKS_ACCOUNT_ID` estiver setado; complementa a busca de usuários do workspace para achar gente ainda não provisionada.

requer Account Admin

Unity Catalog (nativo)

quem pode ver/tagueiar

Não é uma chamada explícita do app — é o próprio UC recusando a operação OBO quando o usuário não tem o grant, sem o app precisar checar nada.

grants usados SELECT/USE, APPLY TAG, ASSIGN

O QUE ESSE APP NÃO USA

Ao contrário dos apps AppKit do time (ex: `rag-pdf-files`), este não tem Lakebase, Volumes nem Model Serving — é Streamlit puro sobre o `databricks-sdk`, e todo o estado próprio vive em tabelas Delta comuns.

07 Notas e observações

Duas coisas que valem registrar para quem for mexer nesse app ou usá-lo em produção.

README E GUIA DE USO DESCREVEM A ESCRITA DE TAGS ERRADO

`README.md` e `docs/07-guia-de-uso.md` afirmam que tags são escritas pelo **service principal**. O código faz o oposto: tags rodam `prefer_user=True` (OBO) em `apply_changes()` — só o comentário roda como SP. O docstring do módulo e `docs/02-arquitetura.md` / `docs/04-permissoes.md` estão corretos e batem com o código; README e o guia de uso ficaram desatualizados desde que o invariante OBO/SP foi formalizado (commit `924c7034`).

GOVERNANÇA FEITA SÓ EM DEV NÃO DURA

O job de sincronização semanal (Fig. 4) sobrescreve comentários e tags em DEV com o que estiver em PROD. Documentação feita na instância DEV é, por design, efêmera — para durar, precisa ser feita na instância PROD.

DOMÍNIOS/SUB-DOMÍNIOS/STEWARDS SÃO SÓ REGISTRO, NÃO TRAVA NADA

Nenhum código de `apply_changes` / `apply_table_comment` / escrita de tag consulta `dominios`, `subdominios` ou `data_stewards`. A atribuição de steward é informativa — hoje não impede nem libera nenhuma edição; quem decide isso continua sendo o grant de UC (tags) e o portão de acesso (comentário).